AI-Powered Fraud Detection System for Online Transactions

Complete Step-by-Step Guide: Running on CMD (Command Prompt), GitHub & Vercel

1. Executive Project Summary

This project implements an enterprise-grade **Dual-Path AI Fraud Detection System** designed for online transactions (UPI, Credit/Debit Cards, Netbanking). It combines tree-based supervised machine learning (XGBoost / Random Forest), deep sequence anomaly detection (LSTM / Autoencoder), and heuristic rules mapped directly to **OWASP API Security Top 10 (2023)** and **PCI DSS v4.0** compliance standards.

Component	Technical Architecture & Specs
Supervised Scoring (45%)	XGBoost / Random Forest with SMOTE balancing tabular features
Deep Sequence Scoring (40%)	LSTM & Autoencoder sliding sequence ($T=10$) for temporal ATO detection
Rule Overlay (15%)	Velocity bursts, device mismatch penalties, high-value heuristics
Decision Policy	3-Tier Policy: **Approve** (<35), **Step-Up 2FA** (35-70), **Block** (>70)
Currency Ecosystem	Calibrated in Indian Rupees (INR ■) for UPI, E-Commerce, Swiggy/Zomato, Gaming

2. System Prerequisites

Make sure your system has the following installed before proceeding:

- **Python 3.10, 3.11, or 3.12:** Download from *python.org*. **Important:** Check the box *'Add Python to PATH'* during installation.
- **Git:** Download and install from *git-scm.com*.
- **Modern Web Browser:** Chrome, Edge, Firefox, or Brave.

3. Running Locally Using Command Prompt (CMD) Inside Project Folder

YES! You can run the entire project directly using Windows Command Prompt (CMD) inside the project folder. In fact, running in CMD inside the folder is the easiest and most recommended method because it avoids PowerShell script execution policy errors.

Step 3.1 — How to Open CMD Directly Inside the Project Folder (The 2-Second Shortcut)

You do NOT need to type long paths or use complex cd commands. Follow this simple shortcut:

1. Open the ai_fraud_detection_system project folder in Windows File Explorer.
2. Click on the **address / location bar** at the top of the folder window (where the folder path is shown).

3. Type **cmd** and press **Enter** on your keyboard.
4. A black Command Prompt window will instantly open, **already located inside your project folder!**

> ***EXPLORER SHORTCUT:*** *Whenever you open CMD via the File Explorer address bar, the prompt automatically displays your project folder path. You are immediately ready to run project commands!*

Step 3.2 — Create & Activate Virtual Environment in CMD

Inside your opened CMD window, run the following commands to create and activate an isolated Python environment:

```
:: Create a virtual environment named 'venv'
python -m venv venv

:: Activate the virtual environment in CMD
venv\Scripts\activate
```

Once activated, you will see **(venv)** appear at the very beginning of your CMD line.

Step 3.3 — Install Project Dependencies

Run the pip installation command inside CMD:

```
pip install -r requirements.txt
```

Step 3.4 — Verify System Tests (Optional & Fast)

Run the automated verification suite to verify that dataset loading and ML models pass 100%:

```
:: Run system test suite (100% passing check)
python test_system.py
```

Step 3.5 — Start the Web Application Server

Launch the Flask application engine from your CMD window:

```
python app.py
```

CMD will display: * Running on http://127.0.0.1:5000.
Open your web browser and navigate to: **http://127.0.0.1:5000** to view the live dashboard!

Step 3.6 — Launch Live Real-Time Payment Stream (In a Second CMD Window)

To simulate real-time live payments arriving at the system every 2.5 seconds:

1. Open the project folder in File Explorer again, click the address bar, type **cmd**, and press **Enter** to open a **second CMD window**.
2. In this second CMD window, activate the venv and run the streaming script:

```
venv\Scripts\activate
python live_checkout_stream.py
```

This window will stream live incoming transactions. Keep both CMD windows open. Switch to your browser at http://127.0.0.1:5000 to observe real-time transaction charts updating dynamically!

4. Web Application User Guide & Modules

The web interface contains five specialized views accessible via the top navigation bar:

Navigation Tab	Functionality & Evaluation Role
Dashboard (/)	Displays total evaluated transactions, approval rates, 2FA challenges, blocked attacks, decision doughnut chart, and fraud categories bar chart.
Live Simulator (/simulator)	Allows entering custom transaction attributes (Amount ■, Category, Device Risk, Velocity, Hour). Features 4 quick preset buttons (UPI Kirana, Card Fraud, ATO, Velocity Burst) to test instant risk evaluation.
Analyst Queue (/queue)	Interactive fraud analyst workstation. Allows manual inspection of flagged transactions with 1-click 'Approve' or 'Block Fraud' override actions.
Model Metrics (/metrics)	Visualizes model performance metrics (Precision, Recall, F1-Score, AUC-PR, ROC-AUC) along with interactive Precision-Recall and ROC curves.
Framework Mapping (/framework)	Detailed enterprise security audit matrix mapping all 5 fraud types to OWASP API Security Top 10 (2023) and PCI DSS v4.0 regulatory controls.

5. Step-by-Step Guide: Uploading Code to GitHub via CMD

You can run all Git upload commands directly inside the same Command Prompt (CMD) window:

Step 5.1 — Create an Empty Repository on GitHub

1. Log in to your GitHub account at **https://github.com**.
2. In the top-right corner, click the **+** icon and select **New repository** (or go to **https://github.com/new**).
3. Enter a **Repository name** (e.g. ai-fraud-detection-system).
4. Select **Public** or **Private**.
5. **CRITICAL:** Leave 'Add a README file', '.gitignore', and 'Choose a license' **UNCHECKED** (the project already has these configured).
6. Click the green **Create repository** button.

Step 5.2 — Run Git Commands in CMD

Inside your CMD window located in the project folder, run the following commands sequentially. Replace <YOUR_GITHUB_USERNAME> and <YOUR_REPOSITORY_NAME> with your actual GitHub username and repo name:

```
:: 1. Configure your Git identity (one-time setup)
git config user.name "Your Name"
git config user.email "your_email@example.com"

:: 2. Initialize and stage all project files
git init
git add .
git commit -m "Initial commit of AI Fraud Detection System"

:: 3. Set branch to main
git branch -M main

:: 4. Link your GitHub repository
git remote add origin https://github.com/<YOUR_GITHUB_USERNAME>/<YOUR_REPOSITORY_NAME>.git

:: 5. Push code to GitHub
git push -u origin main
```

GIT REMOTE UPDATE: *If your project already has a remote configured, simply update it using: git remote set-url origin https://github.com/<YOUR_GITHUB_USERNAME>/<YOUR_REPOSITORY_NAME>.git and then run git push -u origin main.*

6. Step-by-Step Guide: Launching & Hosting on Vercel

The project is pre-configured with vercel.json and an api/index.py serverless WSGI wrapper, making deployment to Vercel completely automated.

Step 6.1 — Log in to Vercel

1. Open your browser and navigate to **https://vercel.com**.
2. Click **Log In** or **Sign Up** and select **Continue with GitHub**.

Step 6.2 — Import Your GitHub Repository

1. On the Vercel Dashboard, click the **Add New...** button in the top-right corner and select **Project**.
2. Under **Import Git Repository**, locate your repository (e.g. ai-fraud-detection-system) and click **Import**.

Step 6.3 — Deploy with 1-Click

1. Leave **Framework Preset** as **Other** (Vercel automatically detects the Python runtime).
2. Leave **Root Directory** as ./.
3. Click the blue **Deploy** button.

Step 6.4 — Access Your Live Web Application

Vercel will build and deploy the Python app in 30–60 seconds. Once completed, Vercel provides a live public HTTPS URL (e.g. https://ai-fraud-detection-system.vercel.app) that anyone can visit globally!

> ***SERVERLESS ARCHITECTURE NOTE:*** *Vercel runs as a serverless function environment. The web dashboard, interactive simulator, analyst queue, and metrics run fully on Vercel. Background continuous looping streams (like live_checkout_stream.py) are meant for local execution on CMD.*

7. Frequently Encountered Questions & Troubleshooting

Scenario / Question	CMD Solution & Explanation
'python' is not recognized in CMD	Python was not added to your Windows PATH during installation. Re-run the Python installer, select **Modify**, and ensure **'Add Python to environment variables'** is checked.
Port 5000 Already in Use	If another app is using port 5000, start Flask on another port in CMD: python app.py --port 5001 or edit port in app.py.
How to stop the server in CMD?	Press **Ctrl + C** in the Command Prompt window to cleanly stop the Flask application or the live streaming simulator.
Git Authentication Error on Push	GitHub requires a **Personal Access Token (PAT)** or GitHub Credential Manager. Generate a token under *GitHub Settings > Developer Settings > Personal Access Tokens* if prompted.

Document Information: *Generated for academic distribution, project assessment, and technical review. Contains complete instructions for autonomous replication, source control, and cloud serverless hosting.*